

Genericitate

① tipuri generice

* beneficii → îmbunătățirea lizibilității codului
↓

siguranță la compilare → nu mai este nevoie de cast explicit.

* instanțierea → nu se pot folosi tipuri primitive
→ se folosesc doar wrapper classes

(Integer, Double, Boolean)

→ autoboxing: conversia automată la obiect și invers (din dată primitivă)

1. push(23); → va converti automat 23 în Integer.

② Semnificația tipurilor de date

→ între <...> sunt convenții pt. a indica rolul parametrului generic generic.

* E → Element → folosit în colecții
ArrayList <E>
Stack <E>

* K → Key → folosit în hărți/dictionare pt. cheie

ex: Map <K, V>

* V → Value → folosit în dictionare pt. valori

* T → Type → un tip generic general, folosit de clase simple

* N → Number, folosit la numere

* S, U → al doilea etc.

③ Metode generice

→ metode definite în clasă, care pot fi sau nu generice, dar care au propriul lor parametru de tip generic.

* sintaxă → param. generic se pune înaintea tipului metode

Exemplu: `public static <T> int count(T[] array)`

* tipul generic al metodei poate fi diferit de cel al clasei.

④ Constrângeri pe tipuri generice

→ se folosește pt. a limita ce tipuri pot fi utilizate ca parametri generici

* cuvântul cheie: extends

* T extends C → T trebuie să fie clasă C sau o subclasă de a ei

* T extends C & I₁ & I₂ → T moștenește clasă C, implementează interfețele I₁ și I₂.

* utilitate: ne permite să apelăm metodele specifice clasei C pe obiectul de tip T.

⑤ Wildcards (Jochuri)

* ? → unbounded wildcard → orice tip

* ? extends T (upper bounded) → acceptă T sau orice subclasă a lui T

* ? super T → T sau orice superclasă a lui T.

⑥ Colectii generice (Java Collections Network)

1. List → ordonată, acceptă duplicate

(List <E>)

- * ArrayList → ca un vector, acces rapid
- * LinkedList → lista dublu înlănțuită → acces rapid, dar inserarea și ștergerea sunt mai rapide.

2. Set → nu ^{poate} conține duplicate

(Set <E>)

* HashSet → nu garantează ordinea elementelor

* TreeSet → elementele sunt păstrate ordonate (alfabetic sau numeric)

3. Map → mapează chei unice la valori

* HashMap → rapid dar neordonat

* TreeMap → sortează elem. după cheie

exemple:

```
List <String> list = new ArrayList <> ();  
list.add("apple"); list.add("capac");
```

```
Map <String, Integer> map = new HashMap <> ();  
map.put("apple", 10);  
map.put("pear", 20);
```

```
Set <String> set = new HashSet <> ();  
set.add("apple"); set.add("banana");  
set.add("apple"); //va conține [banana, apple].
```


* exemplu cu Comparable:

```
class Car implements Comparable <Car> {
```

```
    public int compareTo (Car other) {
```

```
        //sortare alfabetică
```

```
        return this.nume.compareTo (other.nume);
```

```
    }
```

→ în clasa Main (sau în altă clasă):

```
Collections.sort (listaMasini);
```

va folosi Comparable implementat mai sus.

* exemplu cu Comparator:

* vom defini într-o clasă separată acest Comparator pe care îl putem personaliza

```
Comparator public class Main {
```

//definirea Comparatorului

folose

```
Comparator <Car> ComparatorMasini = new Comparator <Car> {
```

```
    @Override  
    public int compare (Car c1, Car c2) {
```

```
        return int. compare (c1.getCai(), c2.getCai());
```

```
    }
```

} ; → se pune ; după definirea comparatorului.

//folosirea comparatorului

```
Collections.sort (listaMasini, comparatorMasini);
```


Exemplu: Comparable & Comparator

```
package org.example;
```

```
class Student implements Comparable<Student> {  
    private String nume;  
    double medie;  
    String cnp;  
  
    public Student(String nume, double medie, String cnp) {  
        this.nume = nume;  
        this.medie = medie;  
        this.cnp = cnp;  
    }
```

@Override

```
    public String toString() {  
        return "Studentul: " + nume + " " + medie + " " + cnp;  
    }
```

```
    public String getCNP() {  
        return cnp;  
    }
```

// COMPARABLE: Definiție logică

// (în interiorul clasei)

```
    /*  
    int-ul returnat de compareTo  
    număr negativ (<0) dacă this < other  
    0, dacă sunt egali  
    număr pozitiv (>0) this > other  
    */
```

@Override

```
    public int compareTo(Student other) {  
        // sortare alfabetică  
        return this.nume.compareTo(other.nume);  
        // dacă am fi vrut INVERS ALFABETIC  
        // return other.nume.compareTo(this.nume);  
    }
```

Studentul implementează
Comparable
(sortare naturală)

comparare naturală
(în interiorul clasei)
după nume

Exemplu in care folosim Comparable si Comparator

```
package org.example;  
import java.util.*;
```

// TIP To Run code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.

```
public class Main {  
    static void main() {  
        List<Student> students = new ArrayList<>();  
        students.add(new Student("Mihai", 9.50, "123"));  
        students.add(new Student("Andrei", 6.70, "22243"));  
        students.add(new Student("Ciprian", 4.58, "987"));
```

→ altă clasă!!

```
        // sa afisam lista initiala  
        System.out.println("-- Lista initiala");
```

```
        for(Student s: students){  
            System.out.println(s);  
        }
```

```
        // facem sortare naturala (foloseste compareTo din clasa)
```

```
        System.out.println("-- SORTARE DUPA NUME");
```

```
        Collections.sort(students);
```

```
        for(Student s: students){  
            System.out.println(s);  
        }
```

→ folosim comparare naturală

→ facem un Comparator nou.

```
        // Comparator cu o logica IN CLASA Main (fara a modifica clasa Student)
```

```
        // se implementeaza intr-o clasa separata
```

```
        Comparator<Student> comparatorMedie = new Comparator<Student>() {
```

```
            @Override
```

```
            public int compare(Student o1, Student o2) {
```

```
                return Double.compare(o1.medie, o2.medie);
```

```
            }
```

```
        };
```

→ comparăm după medie

→ primim comparatorul

```
        Collections.sort(students, comparatorMedie);
```

```
        System.out.println("-- SORTARE DUPA MEDIE CRESCATOR");
```

```
        for(Student s: students){  
            System.out.println(s);  
        }
```


// al doilea comparator personalizat

// Alt comparator cu logica in clasa Main (in afara clasei Student)

definierea comparatorului

```
Comparator<Student> comparatorCNP = new Comparator<Student>() {  
  
    @Override  
    public int compare(Student o1, Student o2) {  
        return o1.getCNP().compareTo(o2.getCNP());  
    }  
};
```

folosirea comparatorului:

```
Collections.sort(students, comparatorCNP);  
System.out.println("-- SORTARE DUPA CNP CRESCATOR");  
for (Student s: students) {  
    System.out.println(s);  
}  
}
```